One Methodology, Two Carriers

Why the research toolkit is built as a twin, and what that buys you

Somewhere in this project two toolkits sit side by side. One is the Claude Code Research Toolkit (**CCRT**), which runs inside Claude Code. The other is the Claude Science Research Toolkit Bundle (**CSRTB**), which runs inside Claude Science. They carry the same research methodology: the same disciplines for planning work, for checking a claim against the record, and for writing so a reader can follow. And yet if you open the two up and compare them line by line, they do not match. One has a guard that halts a bad write before it lands; the other has none. One counts more skills than its sibling. A role that lives as a standalone specialist on one side lives as a paragraph of inline guidance on the other.

A third component rides beside the twin without being a third carrier: the portable planner-kit, which ships inside the Code toolkit at `CCRT/planner-kit/`. It packages the per-project supervisory workflow rather than the shared methodology, and it installs separately: the CCRT once into `~/.claude`, the kit's own installer in each project root that adopts the workflow. The two installers are deliberately separate and designed to be used together.

The natural reaction to a difference is to close it. You find a skill on one side with no twin on the other and you reach for the obvious reading: a gap, port it across. You notice one toolkit carrying more skills than its sibling and you read the shortfall as incompleteness. That reaction is the most expensive mistake you can make with this architecture, because most of those differences are not defects. They are the design. Acting on the wrong reading of a difference either manufactures work that should never be done or, worse, quietly drifts the two toolkits apart until the shared methodology they exist to protect no longer agrees with itself.

This guide explains why the toolkit is built as a twin, and hands you the few ideas that let you read any difference between the two sides correctly. By the end you should be able to look at any mismatch, name what kind of thing it is, and know what, if anything, it asks of you.

Start with what is actually shared. The **methodology** is the intellectual content: the disciplines for decomposing and routing a job, checking a claim against the record instead of asserting it, and making prose stick. That content is single-sourced, authored once and carried twice; the only reason to carry it twice is the platform underneath.

What it is: one methodology on two platforms

The entire architecture rests on a single fact about where the two toolkits run. **Claude Code is a local process.** It has a real shell, files under a folder you can list, hooks that fire on real events and can block an action before it happens, a local audio device, and subagents that run in the same working tree and each hand back one report. **Claude Science is a remote sandbox.** It reaches you only through a browser, it has no hooks and no local audio, it hands work to isolated workers through an asynchronous software interface rather than a shared shell, and it keeps state in an artifact store and a per-specialist memory instead of in files on a disk.

Take one discipline and watch it land differently on each. Both toolkits enforce the same rule: an agent may not claim a result it did not check. On Claude Code that rule is a hook. When an agent

writes a claim that mismatches the record, the hook fires on the write event and blocks it before it lands. Claude Science has no hook surface for a hook to live on, so the identical discipline rides as a prose gate in the planner's instructions, and a background reviewer scores whether the agent honored it. Same job, same intent, two different machines. Each is the discipline expressed in the only atoms its platform offers.

This is why the toolkit is a twin and not something simpler. A single toolkit cannot run on both platforms, because the mechanisms are disjoint and the two runtimes do not co-exist. Two independent toolkits would run fine, but the shared methodology would drift apart the first time a lesson landed on one side and not the other, and no one would notice until the two had diverged past reconciling. Carrying **one methodology on two carriers** threads between those failures: the intellectual content is written once, so it cannot fork, while each mechanism stays native to its platform.

The three tiers

If every difference is not a gap, you need a way to tell the kinds apart. Every load-bearing item in either toolkit classifies into exactly one of three tiers. Call them the **shared tier**, the **split-mechanism tier**, and the **platform-only tier** (TIER-S, TIER-C, and TIER-P in the map that owns the full definitions).

An item in the **shared tier** has the same content and the same kind of carrier on both sides. A change to one should be mirrored to the other, allowing for local vocabulary. The platform-neutral methodology skills live here, the disciplines that carry the same meaning on either platform. An item in the **split-mechanism tier** shares its discipline across both sides but carries it in a different mechanism, because the platforms differ. The claim-checking rule above is exactly this: one discipline, a hook on one side and a prose gate on the other. An item in the **platform-only tier** is meaningful on one platform and inapplicable on the other, so it never crosses. A local-audio alert has nothing to beep through on a browser-only sandbox; a sandbox's org-migration engine has no local object to act on.

The reason there are three and not two is that the middle tier cannot fold into either neighbor without losing something. Fold the split-mechanism tier into the shared tier and you lose the standing warning never to copy its mechanism, so someone eventually pastes a hook into a file that cannot run it. Fold it into the platform-only tier and you lose the real obligation to mirror the change, so a shared lesson lands on one side and silently skips the other. The middle is irreducible, and that is what the tier tag buys you: it is not a label, it is the item's maintenance duty written down. It heads off the two readings that cause the most damage, that "present on both sides" means "should be byte-identical" (false for anything in the split-mechanism tier) and that "absent on one side" means "a gap to fill" (usually false). The authoritative per-item classification lives in the shared-versus-divergent map, and the two rosters live in the roster guide; this guide teaches the model, it does not restate their tables.

The law that protects the twin: never byte-copy

The split-mechanism tier carries one hard rule, and it is worth stating on its own. When a shared discipline changes on one side, you re-express it in the other platform's atoms. You never copy the

mechanism across. The atoms pair up predictably: the Claude Code subagent tool corresponds to the Claude Science delegation interface, a file corresponds to an artifact, a hook corresponds to a piece of prose or a small kernel of logic, and an always-on rule corresponds to a per-specialist clause.

The planner's self-check is the worked example the toolkit itself points to. Claude Science gained a prose gate that has the planner re-read its own turn and emit a marker for the reviewer to score. When that same discipline was carried to Claude Code, the port did not paste the prose across. It added a single line pointing at the hooks that already enforce the check on Code, where a hook is the stronger form. The discipline arrived on both sides; the mechanism did not travel.

The reason the rule is absolute is what a byte-copy actually does. It drags one platform's runtime call into the other platform's file. A Science delegation call pasted into a Code file, or a Code hook pasted into a Science profile, cannot execute where it lands, and the discipline it was meant to carry does not fail loudly. It fails silently, which is the failure the whole tier model exists to prevent.

Reading a difference correctly

Two kinds of difference tempt you most, and both have a disciplined reading. The first is an absence: a skill on one side with no same-named twin on the other. Treat that absence as a question with three answers rather than as a gap. It is in exactly one of three states. The concept may already be **ported under another name or carrier**, since a Science skill can legitimately map to a Code rule or hook rather than to a Code skill; if it is, there is nothing to port, so you check the rules and hooks before concluding anything is missing. A second possibility is that the item is **genuinely platform-only**, dependent on an object the other side does not have. The third is that it is **an open port-candidate**, portable and not yet carried across. You decide which before you act, and an unclassified item defaults to port-candidate, never silently to excluded, because a silent exclusion is indistinguishable from an oversight. Getting this wrong is not free: acting on the wrong reading either builds a duplicate skill, which is a routing hazard because an agent cannot tell which of two same-purpose skills to load, or spends effort re-porting something that already exists under another name.

The second tempting difference is a count. The two rosters are different sizes, and the difference is expected, not a shortfall. Every delta is explained by the tiers already described: platform-only items that live on one side by design, roles carried in a different form on each side, mechanism asymmetries in the split-mechanism tier. Because the deltas are structural, aggregate parity is not a correctness signal. A matched pair of totals would tell you nothing, and a mismatched pair tells you only to classify, not to port. The current counts and the full side-by-side belong to the roster guide, which owns them and keeps them current; they drift as the toolkits grow, so the discipline is to recount from the trees before citing a number rather than trusting one copied into prose.

Why a role sits where it sits

This last difference looks like an inconsistency, but it is not. A methodology role is carried either as a standalone specialist, a Code agent or a Science profile, or as a piece of inline guidance, a skill. The two are not interchangeable, and which form a role takes is a deliberate choice.

The heavier standalone form has to be earned. A role earns it only when it shows a real signal that delegation buys something: its exploration would clutter the main thread and only a distilled summary should return, or it should run on a different model, or it needs a different toolset than its caller, or it is a fresh-eyes pass over work someone else already produced. Absent every one of those signals, the role is skill-shaped, because delegating a decision you must then act on yourself is pure round-trip overhead: you would ask an isolated subagent, wait for a summary, and act inline anyway.

The statistics method-advisor is the worked case. It has none of those signals. It is guidance you consume and act on in your own context as you work, so on Claude Code it is a skill that loads inline. Claude Science has no inline-advisor-skill mechanism at all, so the same content rides a profile there instead. That is the same discipline in the lightest carrier each platform can offer, not a one-to-one gap to be closed. The rule generalizes: pick the lightest form that captures a real signal, and treat a role that takes different forms on the two sides as a carrier choice, never as a defect.

Keeping the twin honest

Two mechanisms hold the twin coherent, and both replace discipline with machinery. They do it for the same reason: discipline leans on memory and attention, the two things code does not have and a working agent cannot rely on in itself.

The first mechanism is the gate. Coherence is enforced structurally rather than by good intentions. You edit the source, never the built artifact: a Science change lands in the source tree and the bundle is rebuilt from it. Fail-closed gates block at the moment of the write rather than flagging it for later. Any count or status is computed at check time rather than written down, because a number typed into prose is a claim about a file that goes stale the moment the file changes, and it will not update itself.

The second mechanism is the flag. A behavior measured on one platform is flagged for re-measurement on the other, never carried across as a fact. The worked case is message-steering. On Claude Code, a message sent to a running subagent was measured to arrive at its next tool call. That delivery timing is a Claude Code measurement. It is not a Claude Science fact, and the Science delivery timing stays unmeasured until someone measures it on Science. Importing the Code figure would be asserting what one system does from a measurement of a different system, which is exactly the move the flag exists to forbid. The async Science interface never even had the "cannot steer mid-run" limitation the Code correction was written for, so the Code figure would answer a question Science never asked.

How the twin stays in sync

Coherence over time comes down to three habits, each aimed at a specific way twins rot. One document is the owner-of-record for what the two sides share and how, and the port-direction questions route to their own named authorities. When you add or change a load-bearing item, you classify it into a tier in the same session, while the platform reasoning is still fresh, because a classification deferred is a classification that later reads as an intentional exclusion, indistinguishable from something simply forgotten. You keep one owner per topic, because two current

copies of a fact drift apart and the divergence is undetectable, which is the worst kind of poison a shared record can carry. And you route every question about what a thing is, where it lives, and why it sits there to the owner-document rather than to your own recollection, with the owner-document winning any disagreement. One of the port-direction authorities, the record of Science skills deliberately not ported back, lives in a historical tree outside this mirror and is not readable from here; where it comes up, that is exactly what to say about it.

What generalizes

Two closing ideas carry the most weight, and then it is worth separating what transfers beyond this project from what does not.

Port direction is bidirectional. Neither platform is the privileged source. Shared content can originate on either side and flow to the other, so a port reads the origin side's own record rather than assuming a fixed direction of travel. The writing-science engine is a real instance: it began on Claude Science, was ported to Claude Code, and its later upgrades now flow back to the Science kernel. Assume a fixed direction and you would overwrite the newer origin with a stale copy, or re-derive something that already exists.

A gate earns trust only after it is shown to catch the defect it exists to catch. You plant the exact fault the gate is meant to stop, confirm the gate fires, and only then rely on it. An untested gate is just another unverified claim, one layer down. The toolkit's own build gates shipped with real bugs their fixtures caught: a checker that crashed on legitimate input, and a false pass that let an empty field through because a null value stringifies to non-empty text. Verify the verifier, then let it speak.

What of all this reaches past this particular toolkit? The generative fact is specific: this twin exists because Claude Code is a local process and Claude Science is a remote sandbox. But the shape around that fact is general. Any system that carries one body of content across two platforms with disjoint mechanisms meets the same three-way split into a portable core, a re-expressed middle, and platform-only pieces. The ban on pasting a mechanism across the divide, the reading of an absence as a question, and the need to enforce coherence in code rather than memory all carry over with it. A cross-platform application sharing a domain core across two operating systems is the same problem in a different dress. The shape has two honest limits. Where the two targets actually share one runtime, a single artifact suffices and no split is warranted. And where one side is a designated upstream, direction is privileged and the bidirectional rule narrows to a one-way flow. This twin is precisely the case where neither carrier is canonical, which is why its discipline has to run in both directions at once.